Práctica 07: Árboles AVL

Equilibrio, rotaciones e inserción · Miniprueba en DomJudge

AED I — Bloque II · Python / C++ · 2 horas

Algoritmia y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

Flujo de la sesión

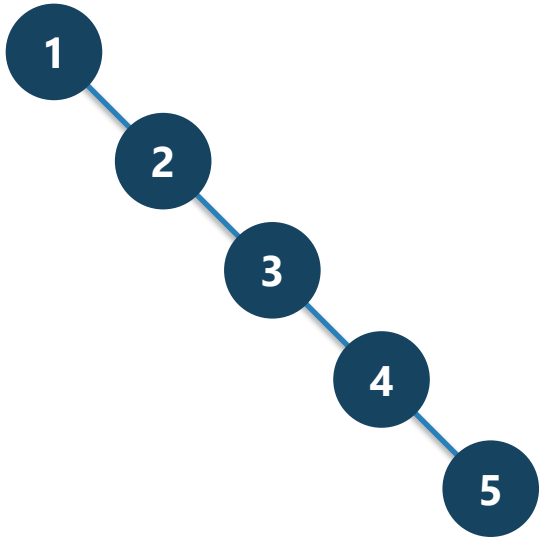
Primera hora: AVL. Segunda hora: miniprueba individual en DomJudge.

Fase	Tiempo	Actividad
Repaso	15 min	Por qué equilibrar, factor de equilibrio, rotaciones y los cuatro casos
Ejercicio 1	10 min	rotar_derecha y rotar_izquierda
Ejercicio 2	15 min	equilibrar: elegir la rotación según los factores de equilibrio
Ejercicio 3	15 min	insertar: inserción de BST + equilibrar a la vuelta de la recursión
Preparación	5 min	Normas de la miniprueba y acceso a DomJudge
Miniprueba	40 min	Problema kmenores: pilas, BST y heaps (individual)
Cierre	5 min	Qué queda para casa: ampliación es_avl y RESPUESTAS.md

¿Por qué equilibrar?

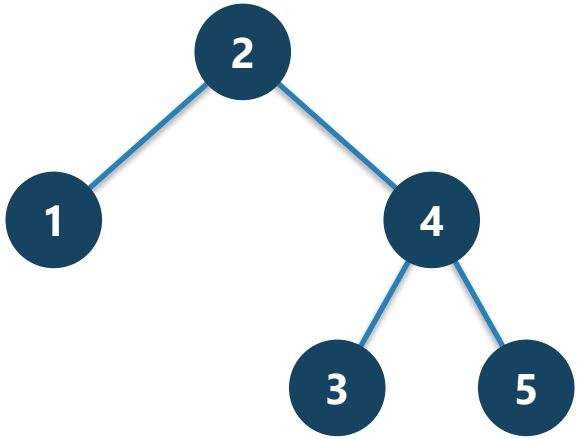
El mismo conjunto de claves puede dar árboles de alturas muy distintas.

BST: INSERTAR 1, 2, 3, 4, 5



altura 4 = n − 1

AVL: LAS MISMAS CLAVES



altura 2

> Con claves ordenadas un BST es una lista: buscar cuesta O(n). Para 1000 claves, altura 999 frente a 9.

Árbol AVL: dos propiedades

1. ES UN BST

- Menores a la izquierda.
- Mayores a la derecha.
- Inorden = claves ordenadas.

2. ESTÁ EQUILIBRADO

- `fe = altura(izq) − altura(der)`
- fe ∈ {−1, 0, +1} en TODOS los nodos.
- Cada nodo guarda su altura.

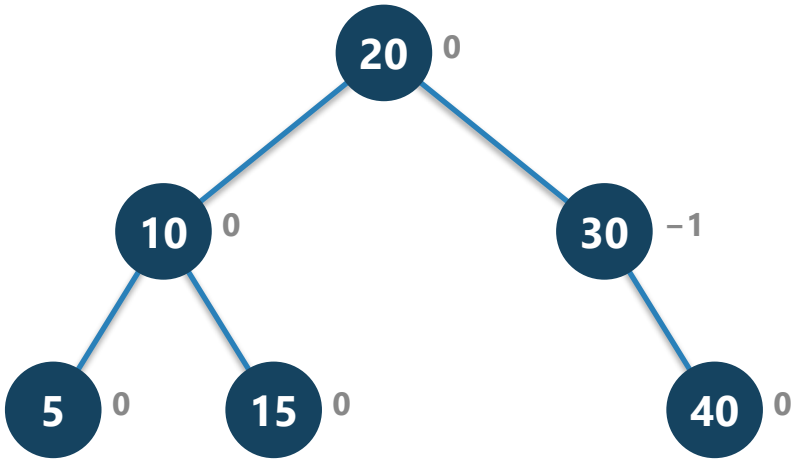
```
altura(vacío) = -1        altura(hoja) = 0
altura(n) = 1 + max(altura(n.izq), altura(n.der))
```

> Altura ≤ 1,44 · $\log_2 n$ ⇒ buscar e insertar en O(log n), también en el peor caso.

Factor de equilibrio

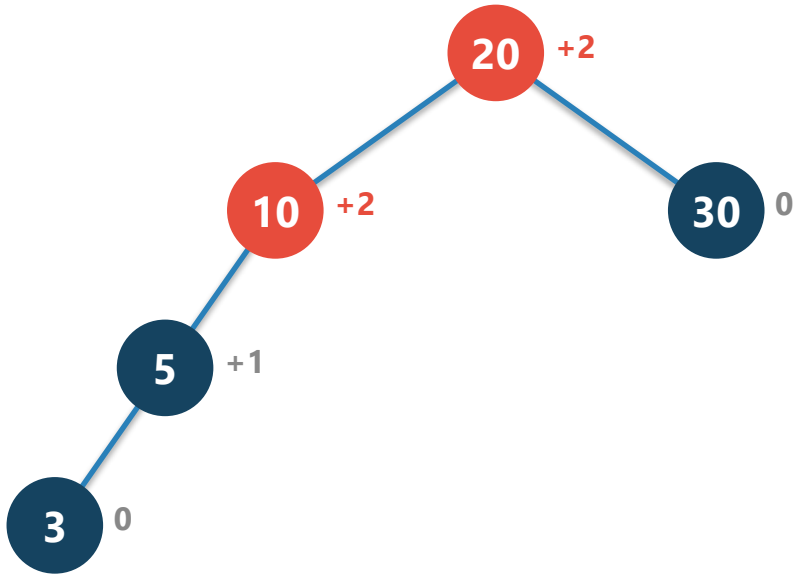
Se calcula en cada nodo con las alturas de sus dos hijos (vacío = −1).

AVL



Las anotaciones grises son el fe de cada nodo.

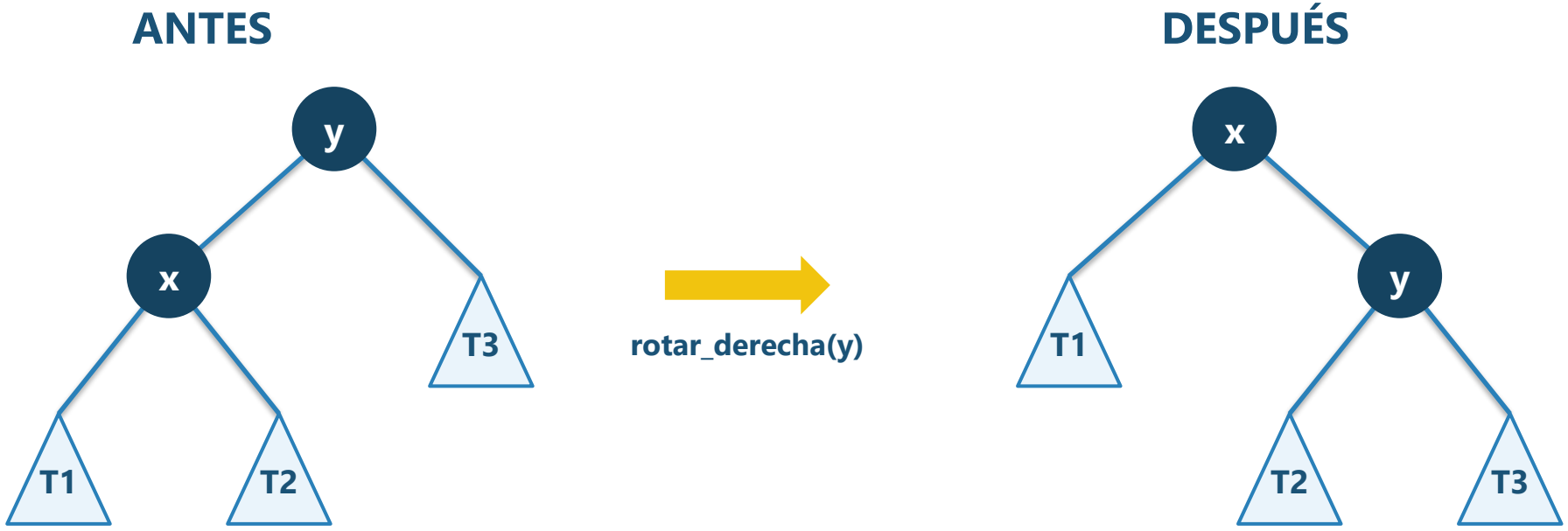
NO ES AVL



> Al insertar se arregla el nodo desequilibrado MÁS BAJO; al rotarlo, los de encima vuelven a estar bien.

Rotación simple a la derecha

Sube el hijo izquierdo x; y baja a su derecha. Solo cambian dos enlaces.




```
x = y.izq
y.izq = x.der
x.der = y
actualizar(y)
actualizar(x)
return x
```

El inorden no cambia (T1 x T2 y T3): sigue siendo un BST. T2 cambia de padre. Coste O(1).

Ejercicio 1: rotaciones

PYTHON

```python
def rotar_derecha(y):
    x = y.izq
    # TODO: 2 enlaces
    actualizar_altura(y)
    actualizar_altura(x)
    return x
```

C++

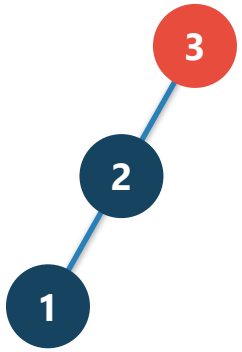
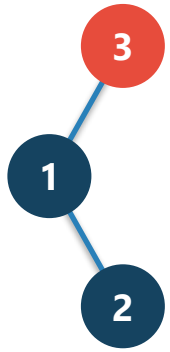
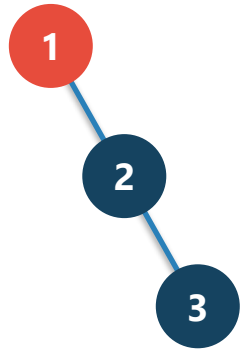
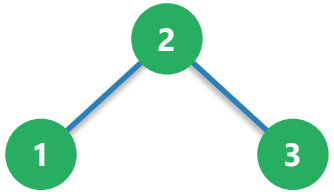
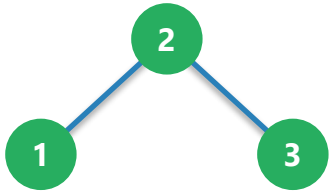
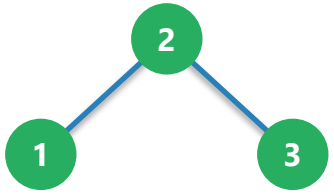
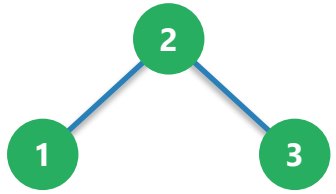
```cpp
Nodo* rotar_derecha(Nodo* y) {
    Nodo* x = y->izq;
    // TODO: 2 enlaces
    actualizar_altura(y);
    actualizar_altura(x);
    return x;
}
```

- rotar_izquierda es el espejo: cambia izq ↔ der y x ↔ y.
- Devuelve la NUEVA raíz del subárbol: quien llama debe reengancharla.

> Primero se actualiza el nodo que queda ABAJO: la altura del de arriba depende de la suya.

Los cuatro casos de desequilibrio

Se mira el fe del nodo (±2) y el de su hijo en ese lado.

IZQUIERDA-IZQUIERDA	IZQUIERDA-DERECHA	DERECHA-DERECHA	DERECHA-IZQUIERDA
			
rotar_derecha(3)	izquierda(1), derecha(3)	rotar_izquierda(1)	derecha(3), izquierda(1)
			

Los cuatro terminan igual: el valor intermedio sube a la raíz. Los casos «en zigzag» necesitan rotación doble.

Ejercicio 2: equilibrar(nodo)

Actualiza la altura del nodo y, si hace falta, rota. Devuelve la nueva raíz.

fe(nodo)	fe del hijo de ese lado	Caso	Qué hacer
+2	fe(izq) ≥ 0	izquierda-izquierda	`return rotar_derecha(nodo)`
+2	fe(izq) < 0	izquierda-derecha	`nodo.izq = rotar_izquierda(nodo.izq)` `return rotar_derecha(nodo)`
−2	fe(der) ≤ 0	derecha-derecha	`return rotar_izquierda(nodo)`
−2	fe(der) > 0	derecha-izquierda	`nodo.der = rotar_derecha(nodo.der)` `return rotar_izquierda(nodo)`
−1, 0, +1	—	equilibrado	`return nodo`

> Error típico: olvidar actualizar_altura(nodo) antes de calcular su fe, o no devolver la nueva raíz.

Ejercicio 3: insertar

Inserción normal de BST y, a la vuelta de cada llamada, equilibrar.

```
def insertar(raiz, clave):
    if raiz is None:
        return Nodo(clave)
    if clave < raiz.clave:
        raiz.izq = insertar(raiz.izq, clave)
    elif clave > raiz.clave:
        raiz.der = insertar(raiz.der, clave)
    else:
        return raiz       # repetida
    return equilibrar(raiz)
```

- Baja como en un BST: O(log n).
- Sube por el mismo camino actualizando alturas.
- Como mucho UNA rotación (simple o doble) por inserción.
- Las repetidas no cambian el árbol.

¡Reenganchar! raiz.izq = insertar(...). Si la rotación cambia la raíz del subárbol, el padre debe enterarse.

Traza: 10, 20, 30, 25, 28, 5

La misma secuencia de la pregunta 1 de RESPUESTAS.md (y de un test).

Inserta	Nodo desequilibrado	Caso	Rotación	Árbol resultante
10	—	—	—	`10`
20	—	—	—	`10(·, 20)`
30	10: fe −2; 20: −1	derecha-derecha	izquierda en 10	`20(10, 30)`
25	—	—	—	`20(10, 30(25, ·))`
28	30: fe +2; 25: −1	izquierda-derecha	izquierda en 25, derecha en 30	`20(10, 28(25, 30))`
5	—	—	—	`20(10(5, ·), 28(25, 30))`

```
print(dibujar(construir([10, 20, 30, 25, 28, 5]), detalles=True))     # utilidades.py
```

Demostración en vivo: VisuAlgo

Abrir en el navegador para ver las rotaciones paso a paso.

https://visualgo.net/en/bst

1. Cambiar a Exploration Mode y elegir «AVL Tree» arriba (no «BST»).
2. Insertar 1, 2, 3, 4, 5, 6, 7 en el BST y después en el AVL: comparar alturas.
3. Insertar 10, 20, 30, 25, 28, 5: anticipar cada rotación antes de verla.
4. Fijarse en las alturas que muestra junto a cada nodo al volver de la recursión.

> Pregunta a la clase: ¿por qué con claves ordenadas el AVL sale perfecto (altura $\lfloor \log_2 n \rfloor$)?

Costes: BST frente a AVL

Operación	BST (peor caso)	AVL	Heap (práctica 6)
Altura	$n - 1$	$\leq 1{,}44 \cdot \log_2 n$	$\lfloor \log_2 n \rfloor$
Buscar una clave	O(n)	O(log n)	O(n)
Insertar	O(n)	O(log n)	O(log n)
Mínimo	O(n)	O(log n)	O(1) en un min-heap
Todas en orden	O(n) inorden	O(n) inorden	O(n log n) extrayendo
Rotar / equilibrar	—	O(1)	—

> Un heap no rota nunca: su forma completa la garantiza el array. Pero no sirve para buscar una clave cualquiera.

Prueba evaluable en DomJudge

Individual · 40 minutos · 2 problemas · Python o C++

QUÉ ENTRA

- Pilas.
- Árboles binarios de búsqueda.
- Heaps.
- Entrada con T casos (práctica 6).

> Dos problemas independientes: construirbst y kmenores. Código base en la página de cada uno.

Antes de enviar: los veredictos

Prueba siempre con el ejemplo del enunciado: python3 kmenores.py < ejemplo.txt

Veredicto	Qué significa	Causa habitual
CORRECT	Todas las pruebas bien	—
WRONG-ANSWER	Alguna salida es distinta	print de depuración, espacios o formato, o el algoritmo
RUN-ERROR	El programa ha terminado con error	Excepción, índice fuera de rango, recursión demasiado profunda
TIMELIMIT	Ha superado el límite de tiempo	Bucle infinito (¿avanzas el nodo?) o algoritmo demasiado lento
COMPILER-ERROR	No compila (C++) o error de sintaxis	Enviar el fichero equivocado o en el lenguaje equivocado

> No cambies la lectura ni la escritura del código base. Cualquier texto de más da WRONG-ANSWER.

¡Manos a la obra!

Rotaciones, equilibrar e insertar · después, la miniprueba